$$(f \uparrow f')(u,v) = \begin{cases} f(u,v) + f'(u,v) - f'(v,u) & \text{if } (u,v) \in E\ , \\ 0 & \text{otherwise}\ . \end{cases} \tag{24.4}$$

The intuition behind this definition follows the definition of the residual network. The flow on (u,v) increases by $f'(u,v)$, but decreases by $f'(v,u)$ because pushing flow on the reverse edge in the residual network signifies decreasing the flow in the original network. Pushing flow on the reverse edge in the residual network is also known as ***cancellation***. For example, suppose that 5 crates of hockey pucks go from u to v and 2 crates go from v to u. That is equivalent (from the perspective of the final result) to sending 3 crates from u to v and none from v to u. Cancellation of this type is crucial for any maximum-flow algorithm.

The following lemma shows that augmenting a flow in G by a flow in G_f yields a new flow in G with a greater flow value.

Lemma 24.1

Let $G = (V,E)$ be a flow network with source s and sink t, and let f be a flow in G. Let G_f be the residual network of G induced by f, and let f' be a flow in G_f. Then the function $f \uparrow f'$ defined in equation (24.4) is a flow in G with value $|f \uparrow f'| = |f| + |f'|$.

Proof We first verify that $f \uparrow f'$ obeys the capacity constraint for each edge in E and flow conservation at each vertex in $V - \{s,t\}$.

For the capacity constraint, first observe that if $(u,v) \in E$, then $c_f(v,u) = f(u,v)$. Because f' is a flow in G_f, we have $f'(v,u) \le c_f(v,u)$, which gives $f'(v,u) \le f(u,v)$. Therefore,

$$\begin{aligned} (f \uparrow f')(u,v) &= f(u,v) + f'(u,v) - f'(v,u) && \text{(by equation (24.4))} \\ &\ge f(u,v) + f'(u,v) - f(u,v) && \text{(because } f'(v,u) \le f(u,v)\text{)} \\ &= f'(u,v) \\ &\ge 0\ . \end{aligned}$$

In addition,

$$\begin{aligned} &(f \uparrow f')(u,v) \\ &\qquad = f(u,v) + f'(u,v) - f'(v,u) && \text{(by equation (24.4))} \\ &\qquad \le f(u,v) + f'(u,v) && \text{(because flows are nonnegative)} \\ &\qquad \le f(u,v) + c_f(u,v) && \text{(capacity constraint)} \\ &\qquad = f(u,v) + c(u,v) - f(u,v) && \text{(definition of } c_f\text{)} \\ &\qquad = c(u,v)\ . \end{aligned}$$

To show that flow conservation holds and that $|f \uparrow f'| = |f| + |f'|$, we first prove the claim that for all $u \in V$, we have

$$\sum_{v \in V} (f \uparrow f')(u,v) - \sum_{v \in V} (f \uparrow f')(v,u)$$
$$= \sum_{v \in V} f(u,v) - \sum_{v \in V} f(v,u) + \sum_{v \in V} f'(u,v) - \sum_{v \in V} f'(v,u) \,. \tag{24.5}$$

Because we disallow antiparallel edges in G (but not in G_f), we know that for each vertex u, there can be an edge (u,v) or (v,u) in G, but never both. For a fixed vertex u, define $V_l(u) = \{v : (u,v) \in E\}$ to be the set of vertices with edges in G leaving u, and define $V_e(u) = \{v : (v,u) \in E\}$ to be the set of vertices with edges in G entering u. We have $V_l(u) \cup V_e(u) \subseteq V$ and, because G contains no antiparallel edges, $V_l(u) \cap V_e(u) = \emptyset$. By the definition of flow augmentation in equation (24.4), only vertices v in $V_l(u)$ can have positive $(f \uparrow f')(u,v)$, and only vertices v in $V_e(u)$ can have positive $(f \uparrow f')(v,u)$. Starting from the left-hand side of equation (24.5), we use this fact and then reorder and group terms, giving

$$\begin{aligned}
&\sum_{v \in V} (f \uparrow f')(u,v) - \sum_{v \in V} (f \uparrow f')(v,u) \\
&\quad = \sum_{v \in V_l(u)} (f \uparrow f')(u,v) - \sum_{v \in V_e(u)} (f \uparrow f')(v,u) \\
&\quad = \sum_{v \in V_l(u)} (f(u,v) + f'(u,v) - f'(v,u)) - \sum_{v \in V_e(u)} (f(v,u) + f'(v,u) - f'(u,v)) \\
&\quad = \sum_{v \in V_l(u)} f(u,v) + \sum_{v \in V_l(u)} f'(u,v) - \sum_{v \in V_l(u)} f'(v,u) \\
&\qquad\qquad - \sum_{v \in V_e(u)} f(v,u) - \sum_{v \in V_e(u)} f'(v,u) + \sum_{v \in V_e(u)} f'(u,v) \\
&\quad = \sum_{v \in V_l(u)} f(u,v) - \sum_{v \in V_e(u)} f(v,u) \\
&\qquad\qquad + \sum_{v \in V_l(u)} f'(u,v) + \sum_{v \in V_e(u)} f'(u,v) - \sum_{v \in V_l(u)} f'(v,u) - \sum_{v \in V_e(u)} f'(v,u) \\
&\quad = \sum_{v \in V_l(u)} f(u,v) - \sum_{v \in V_e(u)} f(v,u) + \sum_{v \in V_l(u) \cup V_e(u)} f'(u,v) - \sum_{v \in V_l(u) \cup V_e(u)} f'(v,u) \,.
\end{aligned} \tag{24.6}$$

In equation (24.6), all four summations can extend to sum over V, since each additional term has value 0. (Exercise 24.2-1 asks you to prove this formally.) Taking all four summations over V, instead of just subsets of V, proves the claim in equation (24.5).

Now we are ready to prove flow conservation for $f \uparrow f'$ and that $|f \uparrow f'| = |f| + |f'|$. For the latter property, let $u = s$ in equation (24.5). Then, we have

$$\begin{aligned}|f \uparrow f'| &= \sum_{v \in V}(f \uparrow f')(s,v) - \sum_{v \in V}(f \uparrow f')(v,s) \\ &= \sum_{v \in V} f(s,v) - \sum_{v \in V} f(v,s) + \sum_{v \in V} f'(s,v) - \sum_{v \in V} f'(v,s) \\ &= |f| + |f'| \ .\end{aligned}$$

For flow conservation, observe that for any vertex u that is neither s nor t, flow conservation for f and f' means that the right-hand side of equation (24.5) is 0, and thus $\sum_{v \in V}(f \uparrow f')(u,v) = \sum_{v \in V}(f \uparrow f')(v,u)$. ■

Augmenting paths

Given a flow network $G = (V, E)$ and a flow f, an ***augmenting path*** p is a simple path from s to t in the residual network G_f. By the definition of the residual network, the flow on an edge (u, v) of an augmenting path may increase by up to $c_f(u, v)$ without violating the capacity constraint on whichever of (u, v) and (v, u) belongs to the original flow network G.

The blue path in Figure 24.4(b) is an augmenting path. Treating the residual network G_f in the figure as a flow network, the flow through each edge of this path can increase by up to 4 units without violating a capacity constraint, since the smallest residual capacity on this path is $c_f(v_2, v_3) = 4$. We call the maximum amount by which we can increase the flow on each edge in an augmenting path p the ***residual capacity*** of p, given by

$$c_f(p) = \min\{c_f(u,v) : (u,v) \text{ is in } p\} \ .$$

The following lemma, which Exercise 24.2-7 asks you to prove, makes the above argument more precise.

Lemma 24.2

Let $G = (V, E)$ be a flow network, let f be a flow in G, and let p be an augmenting path in G_f. Define a function $f_p : V \times V \to \mathbb{R}$ by

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p \ , \\ 0 & \text{otherwise} \ . \end{cases} \tag{24.7}$$

Then, f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$. ■

The following corollary shows that augmenting f by f_p produces another flow in G whose value is closer to the maximum. Figure 24.4(c) shows the result of augmenting the flow f from Figure 24.4(a) by the flow f_p in Figure 24.4(b), and Figure 24.4(d) shows the ensuing residual network.

Corollary 24.3

Let $G = (V, E)$ be a flow network, let f be a flow in G, and let p be an augmenting path in G_f. Let f_p be defined as in equation (24.7), and suppose that f is augmented by f_p. Then the function $f \uparrow f_p$ is a flow in G with value $|f \uparrow f_p| = |f| + |f_p| > |f|$.

Proof Immediate from Lemmas 24.1 and 24.2. ■

Cuts of flow networks

The Ford-Fulkerson method repeatedly augments the flow along augmenting paths until it has found a maximum flow. How do we know that when the algorithm terminates, it has actually found a maximum flow? The max-flow min-cut theorem, which we will prove shortly, tells us that a flow is maximum if and only if its residual network contains no augmenting path. To prove this theorem, though, we must first explore the notion of a cut of a flow network.

A ***cut*** (S, T) of flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$. (This definition is similar to the definition of "cut" that we used for minimum spanning trees in Chapter 21, except that here we are cutting a directed graph rather than an undirected graph, and we insist that $s \in S$ and $t \in T$.) If f is a flow, then the ***net flow*** $f(S, T)$ across the cut (S, T) is defined to be

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) . \tag{24.8}$$

The ***capacity*** of the cut (S, T) is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v) . \tag{24.9}$$

A ***minimum cut*** of a network is a cut whose capacity is minimum over all cuts of the network.

You probably noticed that the definitions of flow across a cut and capacity of a cut differ in that flow counts edges going in both directions across the cut, but capacity counts only edges going from the source side of the cut toward the sink side. This asymmetry is intentional and important. The reason for this difference will become apparent later in this section.

Figure 24.5 shows the cut $(\{s, v_1, v_2\}, \{v_3, v_4, t\})$ in the flow network of Figure 24.1(b). The net flow across this cut is

$$\begin{aligned} f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) &= 12 + 11 - 4 \\ &= 19 , \end{aligned}$$

and the capacity of this cut is

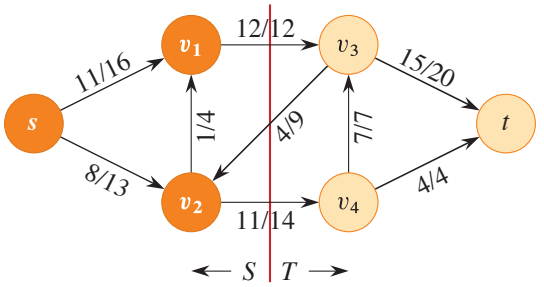

Figure 24.5 A cut (S, T) in the flow network of Figure 24.1(b), where $S = \{s, v_1, v_2\}$ and $T = \{v_3, v_4, t\}$. The vertices in S are orange, and the vertices in T are tan. The net flow across (S, T) is $f(S, T) = 19$, and the capacity is $c(S, T) = 26$.

$$\begin{aligned} c(v_1, v_3) + c(v_2, v_4) &= 12 + 14 \\ &= 26 . \end{aligned}$$

The following lemma shows that, for a given flow f, the net flow across any cut is the same, and it equals $|f|$, the value of the flow.

Lemma 24.4
Let f be a flow in a flow network G with source s and sink t, and let (S, T) be any cut of G. Then the net flow across (S, T) is $f(S, T) = |f|$.

Proof For any vertex $u \in V - \{s, t\}$, rewrite the flow-conservation condition as

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0 . \tag{24.10}$$

Taking the definition of $|f|$ from equation (24.1) and adding the left-hand side of equation (24.10), which equals 0, summed over all vertices in $S - \{s\}$, gives

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right) .$$

Expanding the right-hand summation and regrouping terms yields

$$\begin{aligned} |f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\ &= \sum_{v \in V} \left(f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left(f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\ &= \sum_{v \in V} \sum_{u \in S} f(u, v) - \sum_{v \in V} \sum_{u \in S} f(v, u) . \end{aligned}$$

Because $V = S \cup T$ and $S \cap T = \emptyset$, splitting each summation over V into summations over S and T gives

$$\begin{aligned}
|f| &= \sum_{v \in S} \sum_{u \in S} f(u,v) + \sum_{v \in T} \sum_{u \in S} f(u,v) - \sum_{v \in S} \sum_{u \in S} f(v,u) - \sum_{v \in T} \sum_{u \in S} f(v,u) \\
&= \sum_{v \in T} \sum_{u \in S} f(u,v) - \sum_{v \in T} \sum_{u \in S} f(v,u) \\
&\qquad + \left(\sum_{v \in S} \sum_{u \in S} f(u,v) - \sum_{v \in S} \sum_{u \in S} f(v,u) \right) .
\end{aligned}$$

The two summations within the parentheses are actually the same, since for all vertices $x, y \in S$, the term $f(x,y)$ appears once in each summation. Hence, these summations cancel, yielding

$$\begin{aligned}
|f| &= \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u) \\
&= f(S,T) . \qquad \blacksquare
\end{aligned}$$

A corollary to Lemma 24.4 shows how cut capacities bound the value of a flow.

Corollary 24.5
The value of any flow f in a flow network G is bounded from above by the capacity of any cut of G.

Proof Let (S,T) be any cut of G and let f be any flow. By Lemma 24.4 and the capacity constraint,

$$\begin{aligned}
|f| &= f(S,T) \\
&= \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u) \\
&\le \sum_{u \in S} \sum_{v \in T} f(u,v) \\
&\le \sum_{u \in S} \sum_{v \in T} c(u,v) \\
&= c(S,T) . \qquad \blacksquare
\end{aligned}$$

Corollary 24.5 yields the immediate consequence that the value of a maximum flow in a network is bounded from above by the capacity of a minimum cut of the network. The important max-flow min-cut theorem, which we now state and prove, says that the value of a maximum flow is in fact equal to the capacity of a minimum cut.

Theorem 24.6 (Max-flow min-cut theorem)
If f is a flow in a flow network $G = (V, E)$ with source s and sink t, then the following conditions are equivalent:

1. f is a maximum flow in G.
2. The residual network G_f contains no augmenting paths.
3. $|f| = c(S, T)$ for some cut (S, T) of G.

Proof (1) $\Rightarrow$ (2): Suppose for the sake of contradiction that f is a maximum flow in G but that G_f has an augmenting path p. Then, by Corollary 24.3, the flow found by augmenting f by f_p, where f_p is given by equation (24.7), is a flow in G with value strictly greater than $|f|$, contradicting the assumption that f is a maximum flow.

(2) $\Rightarrow$ (3): Suppose that G_f has no augmenting path, that is, that G_f contains no path from s to t. Define

$$S = \{v \in V : \text{there exists a path from } s \text{ to } v \text{ in } G_f\}$$

and $T = V - S$. The partition (S, T) is a cut: we have $s \in S$ trivially and $t \notin S$ because there is no path from s to t in G_f. Now consider a pair of vertices $u \in S$ and $v \in T$. If $(u, v) \in E$, we must have $f(u, v) = c(u, v)$, since otherwise $(u, v) \in E_f$, which would place v in set S. If $(v, u) \in E$, we must have $f(v, u) = 0$, because otherwise $c_f(u, v) = f(v, u)$ would be positive and we would have $(u, v) \in E_f$, which again would place v in S. Of course, if neither (u, v) nor (v, u) belongs to E, then $f(u, v) = f(v, u) = 0$. We thus have

$$\begin{aligned} f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 \\ &= c(S, T) \, . \end{aligned}$$

By Lemma 24.4, therefore, $|f| = f(S, T) = c(S, T)$.

(3) $\Rightarrow$ (1): By Corollary 24.5, $|f| \leq c(S, T)$ for all cuts (S, T). The condition $|f| = c(S, T)$ thus implies that f is a maximum flow. ■

The basic Ford-Fulkerson algorithm

Each iteration of the Ford-Fulkerson method finds *some* augmenting path p and uses p to modify the flow f. As Lemma 24.2 and Corollary 24.3 suggest, replacing f by $f \uparrow f_p$ produces a new flow whose value is $|f| + |f_p|$. The procedure FORD-FULKERSON on the next page implements the method by updating the flow

attribute $(u, v).f$ for each edge $(u, v) \in E$.[1] It assumes implicitly that $(u, v).f = 0$ if $(u, v) \notin E$. The procedure also assumes that the capacities $c(u, v)$ come with the flow network, and that $c(u, v) = 0$ if $(u, v) \notin E$. The procedure computes the residual capacity $c_f(u, v)$ in accordance with the formula (24.2). The expression $c_f(p)$ in the code is just a temporary variable that stores the residual capacity of the path p.

```
FORD-FULKERSON(G, s, t)
1  for each edge (u, v) ∈ G.E
2      (u, v).f = 0
3  while there exists a path p from s to t in the residual network G_f
4      c_f(p) = min {c_f(u, v) : (u, v) is in p}
5      for each edge (u, v) in p
6          if (u, v) ∈ G.E
7              (u, v).f = (u, v).f + c_f(p)
8          else (v, u).f = (v, u).f − c_f(p)
9  return f
```

The FORD-FULKERSON procedure simply expands on the FORD-FULKERSON-METHOD pseudocode given earlier. Figure 24.6 shows the result of each iteration in a sample run. Lines 1–2 initialize the flow f to 0. The **while** loop of lines 3–8 repeatedly finds an augmenting path p in G_f and augments flow f along p by the residual capacity $c_f(p)$. Each residual edge in path p is either an edge in the original network or the reversal of an edge in the original network. Lines 6–8 update the flow in each case appropriately, adding flow when the residual edge is an original edge and subtracting it otherwise. When no augmenting paths exist, the flow f is a maximum flow.

Analysis of Ford-Fulkerson

The running time of FORD-FULKERSON depends on the augmenting path p and how it's found in line 3. If the edge capacities are irrational numbers, it's possible to choose the augmenting path so that the algorithm never terminates: the value of the flow increases with successive augmentations, but never converges to the maximum flow value. The good news is that if the algorithm finds the augmenting path by using a breadth-first search (which we saw in Section 20.2), it runs in

[1] Recall from Section 20.1 that we represent an attribute f for edge (u, v) with the same style of notation—$(u, v).f$—that we use for an attribute of any other object.

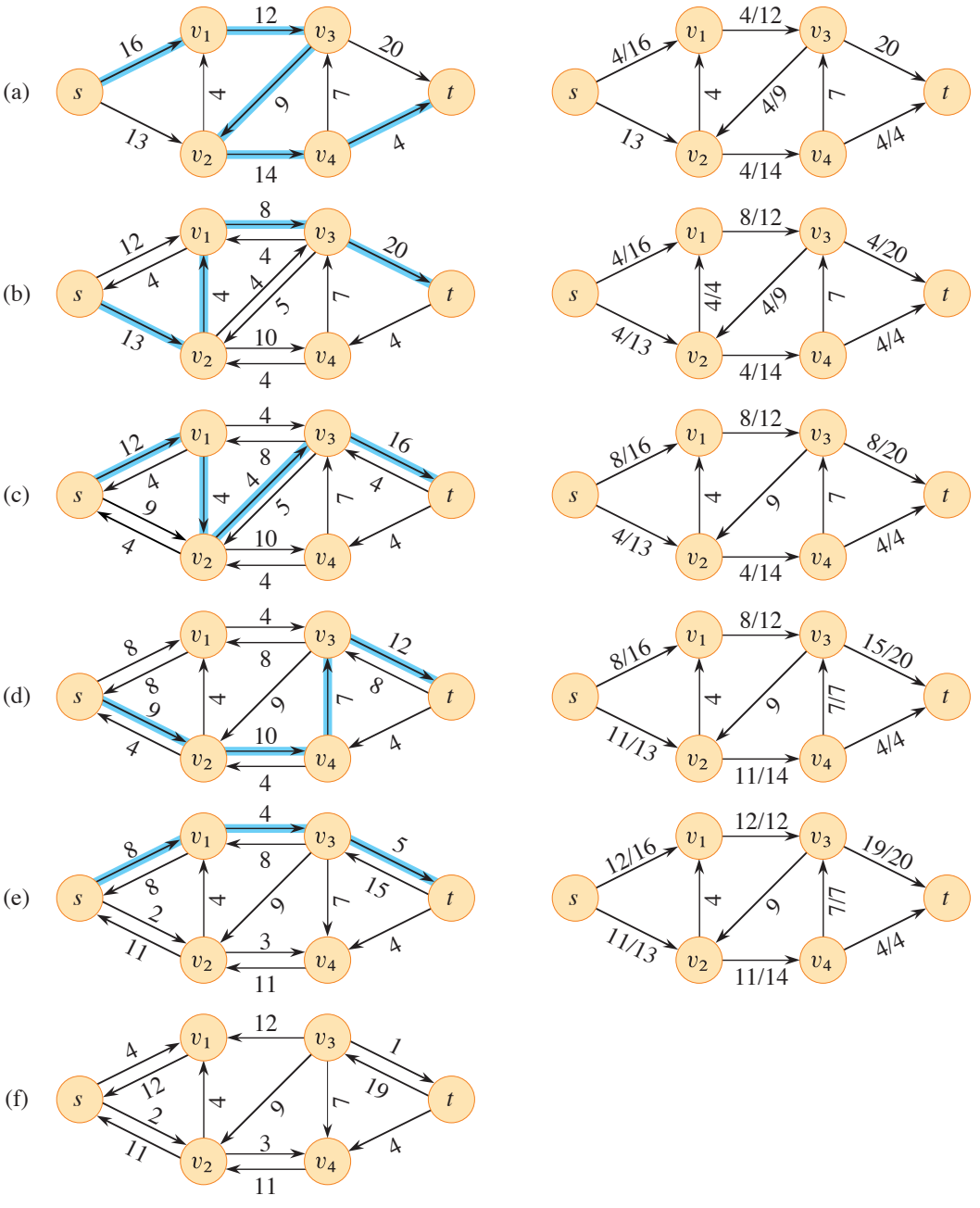

Figure 24.6 The execution of the basic Ford-Fulkerson algorithm. **(a)–(e)** Successive iterations of the **while** loop. The left side of each part shows the residual network G_f from line 3 with a blue augmenting path p. The right side of each part shows the new flow f that results from augmenting f by f_p. The residual network in (a) is the input flow network G. **(f)** The residual network at the last **while** loop test. It has no augmenting paths, and the flow f shown in (e) is therefore a maximum flow. The value of the maximum flow found is 23.

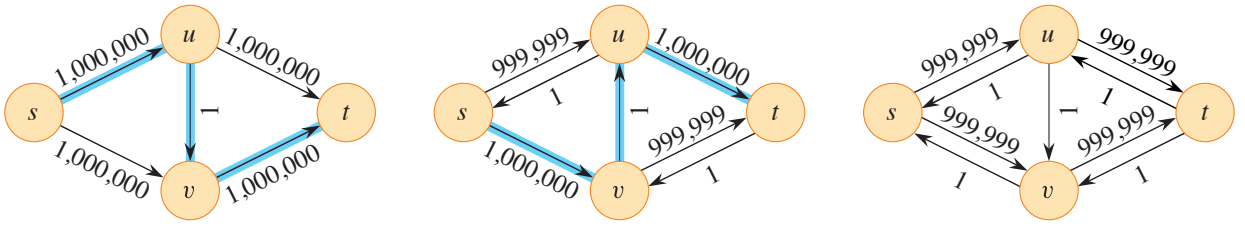

Figure 24.7 **(a)** A flow network for which FORD-FULKERSON can take $\Theta(E\ |f^*|)$ time, where f^* is a maximum flow, shown here with $|f^*| = 2{,}000{,}000$. The blue path is an augmenting path with residual capacity 1. **(b)** The resulting residual network, with another augmenting path whose residual capacity is 1. **(c)** The resulting residual network.

polynomial time. Before proving this result, we obtain a simple bound for the case in which all capacities are integers and the algorithm finds any augmenting path.

In practice, the maximum-flow problem often arises with integer capacities. If the capacities are rational numbers, an appropriate scaling transformation can make them all integers. If f^* denotes a maximum flow in the transformed network, then a straightforward implementation of FORD-FULKERSON executes the **while** loop of lines 3–8 at most $|f^*|$ times, since the flow value increases by at least 1 unit in each iteration.

A good implementation should perform the work done within the **while** loop efficiently. It should represent the flow network $G = (V, E)$ with the right data structure and find an augmenting path by a linear-time algorithm. Let's assume that the implementation keeps a data structure corresponding to a directed graph $G' = (V, E')$, where $E' = \{(u, v) : (u, v) \in E \text{ or } (v, u) \in E\}$. Edges in the network G are also edges in G', making it straightforward to maintain capacities and flows in this data structure. Given a flow f on G, the edges in the residual network G_f consist of all edges (u, v) of G' such that $c_f(u, v) > 0$, where c_f conforms to equation (24.2). The time to find a path in a residual network is therefore $O(V + E') = O(E)$ using either depth-first search or breadth-first search. Each iteration of the **while** loop thus takes $O(E)$ time, as does the initialization in lines 1–2, making the total running time of the FORD-FULKERSON algorithm $O(E\ |f^*|)$.

When the capacities are integers and the optimal flow value $|f^*|$ is small, the running time of the Ford-Fulkerson algorithm is good. Figure 24.7(a) shows an example of what can happen on a simple flow network for which $|f^*|$ is large. A maximum flow in this network has value 2,000,000: 1,000,000 units of flow traverse the path $s \to u \to t$, and another 1,000,000 units traverse the path $s \to v \to t$. If the first augmenting path found by FORD-FULKERSON is $s \to u \to v \to t$, shown